Practice Question Set

1. **What is a union in C, and what is its core difference compared to a structure?**
2. **Write the syntax definition for a union named abc containing an integer a and a character b.**
3. **How does the compiler determine the total memory size allocated to a union variable?**
4. **What happens to the previously stored data when a new member of a union is assigned a value?**
5. **Given a system where int takes 4 bytes, char takes 1 byte, and float takes 4 bytes, what is the evaluated size of union abc { int a; char b; float c; };?**
6. **What addresses will be printed if you inspect the memory locations of two different members of the same union instance?**
7. **What is the "Rule of Thumb" to successfully retrieve and use meaningful data from a union variable?**
8. **Can all members of a union be safely accessed at the exact same time? Explain.**
9. **In a union memory layout, at which byte index do all members begin their memory allocation?**
10. **Summarize the memory layout differences between struct abc and union abc when both contain int a and char b.**

Important Interview/Viva Questions

11. **Why do all members of a union share the exact same starting memory address? Explain using the memory mapping concept.**

12. Trace the output of the following program snippet and explain why garbage values are printed for u.a and u.b:

```
union abc {
   int a;
   char b;
   float c;
} u;
u.a = 1;
u.b = 'a';
u.c = 9.2;
printf("a = %d\n", u.a);printf("b = %c\n", u.b);printf("c
= %f\n", u.c);
```

13. Create a side-by-side comparison table summarizing the differences between a struct and a union based on Keyword, Memory Layout, Total Size, Member Alteration, and Simultaneous Use.

14. What is the primary real-world advantage of using a union instead of a struct in memory-constrained programming?

15. **If a union contains a large array (e.g., char buffer[100];) alongside an int x; and a double y;, what will be the total memory allocated to the union? Explain.**

Answers

1. What is a union in C, and what is its core difference compared to a structure?

Answer: A union is a user-defined data type in C similar to a structure. The core difference is that while each structure member receives its own unique, isolated

memory slot, all members of a union share the exact same memory location. Consequently, a union variable can hold only one valid member value at any given time.

2. Write the syntax definition for a union named abc containing an integer a and a character b.

Answer:

```
C
union abc
{
   int a;
   char b;
};
```

3. How does the compiler determine the total memory size allocated to a union variable?

Answer: Because all union members share memory, the compiler allocates a single block equal to the size of its single largest data member, rather than summing up the individual sizes of all members.

4. What happens to the previously stored data when a new member of a union is assigned a value?

Answer: Modifying one member directly overwrites and corrupts the data stored in the shared memory location by other members. Only the most recently assigned member will contain valid data.

5. Given a system where int takes 4 bytes, char takes 1 byte, and float takes 4 bytes, what is the evaluated size of union abc { int a; char b; float c; };?

Answer: The total size is **4 bytes**. This is because the largest members (float and int) consume 4 bytes, so the union allocates a shared 4-byte memory block.

6. What addresses will be printed if you inspect the memory locations of two different members of the same union instance?

Answer: Both members will return the **exact same starting memory address** (e.g., if &u.a is 6295616, &u.b will also be 6295616).

7. What is the "Rule of Thumb" to successfully retrieve and use meaningful data from a union variable?

Answer: To extract meaningful data, you must access and read a member variable immediately after assigning a value to it, before any other member allocations or assignments take place.

8. Can all members of a union be safely accessed at the exact same time? Explain.

Answer: No. Because all members share the same overlapping memory footprint, only one member can be accessed safely at a time. Reading another member simultaneously will result in corrupted garbage values.

9. In a union memory layout, at which byte index do all members begin their memory allocation?

Answer: All union members share and point to **byte index 0** of the allocated block.

10. Summarize the memory layout differences between struct abc and union abc when both contain int a and char b.

Answer:

> **struct abc:** Allocates sequential, independent blocks (a gets 4 bytes, b gets 1 byte + padding), leading to a total size of 5 bytes or more, where each variable has a unique memory address.
>
> **union abc:** Allocates an overlapping shared block where both a and b point to the exact same starting address, yielding a total size equal to the largest member (4 bytes).

11. Why do all members of a union share the exact same starting memory address? Explain using the memory mapping concept.

Answer: Unions are engineered specifically to save memory when a data structure only needs to store one piece of information at a time from a set of possible types. Instead of carving out separate sequential memory slots for every declared

parameter like a struct does, the compiler maps every member's base offset to byte index 0 of a single, shared memory block sized to fit the largest member.

12. Trace the output of the program snippet and explain why garbage values are printed for u.a and u.b.

Answer:

Trace:

u.a = 1; places the integer 1 in the 4-byte block.

u.b = 'a'; writes ASCII value 97 directly over byte index 0, corrupting u.a.

u.c = 9.2; overwrites the entire 4-byte block with the floating-point representation of 9.2, corrupting both u.a and u.b.

Output:

a = [Garbage value] (data overwritten by float bits)

b = [Garbage value] (data overwritten by float bits)

c = 9.200000 (Valid, as u.c was the last active assignment)

13. Create a side-by-side comparison table summarizing the differences between a struct and a union.

Answer:

Feature	**Structure (struct)**	**Union (union)**
Keyword	struct	union
Memory Layout	Every member gets an independent address.	All members share the same starting address.
Total Size	Sum of sizes of all	Size of the single largest

Feature	Structure (struct)	Union (union)
	members (+ padding bytes).	member.
Member Alteration	Changing one member does not affect others.	Changing one member alters/corrupts all other members.
Simultaneous Use	All members can be accessed concurrently.	Only one member can be accessed safely at a time.

14. What is the primary real-world advantage of using a union instead of a struct in memory-constrained programming?

Answer: The primary advantage is **drastic memory conservation**. In systems with limited RAM (such as embedded systems or microcontroller drivers), if a data model only requires one attribute to be active at any given moment, a union avoids wasting memory by sharing a single block across multiple fields rather than allocating space for all of them.

15. If a union contains a large array (e.g., char buffer[100];) alongside an int x; and a double y;, what will be the total memory allocated to the union? Explain.

Answer: The total memory allocated will be **100 bytes**. This is because char buffer[100] requires 100 bytes, int x requires 4 bytes, and double y requires 8 bytes. Since the compiler allocates a block equal to the size of the single largest member, it chooses the 100-byte array so that any member can safely reside in that shared memory footprint.